NEXTSTAT

Consultoría en Ciencia de Datos e Inteligencia Artificial

ARQUITECTURA DE SOLUCIÓN

NexTask AI Agent

Stack Óptimo para Antigravity · Google ADK + Supabase + Upstash Redis

Documento	ARQ-NexTask-v1.0
Tipo	Arquitectura de Solución + Guía Antigravity
Sistema	NexTask AI Agent (Productividad via Telegram)
Stack	**Google ADK · Supabase · Upstash Redis · FastAPI**
Estado	**✅ Reemplaza n8n completamente — Optimizado para Antigravity**
Fecha	Marzo 2026

1. ¿POR QUÉ N8N NO ES LA OPCIÓN ÓPTIMA?

Antes de proponer la alternativa, es clave entender el problema de fondo. n8n es excelente para automatización de procesos empresariales lineales — pero NexTask no es una automatización lineal, es un agente conversacional con estado.

n8n fue diseñado para...	NexTask necesita...
Flujos lineales: A → B → C	Bucles de razonamiento: clasifica → decide → confronta → responde
Eventos independientes (cada ejecución es nueva)	Memoria persistente entre mensajes del mismo usuario
Configuración visual nodo por nodo (manual)	Código declarativo que Antigravity puede generar automáticamente
14 workflows separados para este PRD	1 solo agente Python modular
Debugging: ver logs nodo por nodo visualmente	Trazabilidad end-to-end con stack traces en código
Gestión del estado: guardar/recuperar de Notion manualmente (2-3 llamadas API por mensaje)	SessionService nativo: el estado vive en el framework, 0 llamadas extra

🔑 **Conclusión clave sobre Antigravity**

Antigravity genera código estructurado (Python/YAML), no JSON de workflows visuales. Un agente escrito en Python con Google ADK es el formato ideal para que Antigravity lo genere, modifique e itere. n8n requeriría que Antigravity genere un esquema JSON propietario de 14 archivos — mucho más frágil.

2. STACK RECOMENDADO — COMPONENTE POR COMPONENTE

2.1 Visión General del Stack

Capa	Componente	Reemplaza a	Justificación
Orquestador	**Google ADK**	n8n (14 workflows)	Framework nativo para agentes Gemini. Elimina workflows manuales. Código Python generado por Antigravity.
LLM	**Gemini 2.0 Flash**	(sin cambio — ya en PRD)	Mismo LLM del PRD original. Free tier 1M tokens/día. Nativo con ADK.
Base de Datos	**Supabase**	Notion como BD primaria	PostgreSQL gratuito. Latencia <50ms vs 400-800ms Notion. Queries complejas. Sin rate limit.
Memoria conv.	**Upstash Redis**	(gap del PRD — nuevo)	Memoria conversacional entre mensajes. Free tier 10K req/día. TTL automático.
API Webhook	**FastAPI + Cloud Run**	n8n Webhook node	Recibe mensajes de Telegram. Serverless. Escala a 0 (costo $0 en uso bajo).
Interfaz usuario	**Telegram Bot API**	(sin cambio — ya en PRD)	Sin cambio. Todo el código de captura se mantiene igual.
Calendario	**Google Calendar API**	(sin cambio — ya en PRD)	Sin cambio. Se integra como Tool de ADK.
Vista visual	**Notion (solo UI)**	Notion como BD — reducido	Notion se mantiene como dashboard de visualización para el usuario. No como BD de operaciones.
Scheduler	**Cloud Scheduler**	n8n Cron triggers	Dispara check-in matutino (7:30 AM) y WF-09 confrontación (c/6h). Free tier ilimitado.

💰 **Costo total de infraestructura: $0/mes**

Todos los componentes tienen free tier suficiente para un usuario individual: Google ADK (open-source), Gemini Flash (1M tokens/día gratis), Supabase (500MB gratis), Upstash Redis (10K req/día gratis), Cloud Run (2M invocaciones/mes gratis), Cloud Scheduler (3 jobs gratis).

3. ARQUITECTURA COMPLETA

3.1 Diagrama de Flujo Principal

ARQUITECTURA NEZTASK — GOOGLE ADK

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                        NEZTASK — ARQUITECTURA ADK                            │
└──────────────────────────────────────────────────────────────────────────────┘

  [Ricardo - Telegram]
        │
        │ POST /webhook  (texto, voz, botón)
        ▼
  ┌──────────────┐
  │  Cloud Run   │  ←── FastAPI app.py (3 endpoints)
  │  FastAPI     │       /webhook  /healthz  /scheduler
  └──────────────┘
        │
        │ message + chat_id
        ▼
  ┌──────────────────────────────────────────────────────────────────┐
  │                        GOOGLE ADK AGENT                          │
  │                                                                  │
  │   Runner                                                         │
  │   ├── Agent: NexTaskAgent (instrucciones + personalidad)         │
  │   │      └── LLM: Gemini 2.0 Flash                              │
  │   │                                                              │
  │   ├── SessionService → Upstash Redis (memoria conv.)             │
  │   │       key: session:{chat_id}  TTL: 24h                       │
  │   │                                                              │
  │   └── Tools (function calling nativo):                           │
  │         ├── create_task(titulo, categoria, deadline)             │
  │         ├── get_tasks(filter, sort_by)                           │
  │         ├── update_task(id, campo, valor)                        │
  │         ├── save_hallazgo(texto, contexto)                       │
  │         ├── get_daily_plan(energia_nivel)                        │
  │         ├── sync_calendar(tarea_id)                              │
  │         ├── transcribe_audio(telegram_file_id)                   │
  │         └── send_telegram(chat_id, mensaje, botones)             │
  └──────────────────────────────┬───────────────────────────────────┘
                                 │
                                 │
        ┌────────────────────────┼────────────────────────┐
        ▼                        ▼                        ▼
  ┌──────────────┐     ┌────────────────────┐     ┌────────────────────┐
  │ Supabase     │     │  Google Calendar   │     │  Notion API        │
  │ PostgreSQL   │     │  API               │     │  (solo escritura   │
  │ (BD ops)     │     │  (sincronización)  │     │   de hallazgos)    │
  └──────────────┘     └────────────────────┘     └────────────────────┘


  ┌──────────────────────────────────────────────────────────────────┐
  │  Cloud Scheduler (triggers automáticos)                          │
  │  ├── 07:30 AM diario → POST /scheduler?event=morning_checkin     │
  │  ├── cada 6h         → POST /scheduler?event=confrontacion       │
```

```
   │   └── Domingo 18:00 → POST /scheduler?event=weekly_report    │
   └──────────────────────────────────────────────────────────────┘
```

3.2 Estructura del Proyecto (para Antigravity)

Esta estructura modular es exactamente lo que Antigravity necesita para generar código por partes, sin dependencias cruzadas. Cada módulo es independiente y tiene una sola responsabilidad.

```
ESTRUCTURA DEL PROYECTO — MÓDULOS INDEPENDIENTES
neztask/
├── main.py                    # FastAPI app — punto de entrada único
├── agent/
│   ├── __init__.py
│   ├── neztask_agent.py       # Definición del agente ADK (LLM + Tools +
instrucciones)
│   ├── instructions.py        # System prompt en español (separado del
código)
│   └── runner.py              # Lógica de ejecución del agente por mensaje
├── tools/
│   ├── __init__.py
│   ├── task_tools.py          # create_task, get_tasks, update_task,
complete_task
│   ├── hallazgo_tools.py      # save_hallazgo, get_hallazgos
│   ├── planning_tools.py      # get_daily_plan, get_weekly_summary
│   ├── calendar_tools.py      # sync_calendar, create_event
│   ├── audio_tools.py         # transcribe_audio (Gemini nativo)
│   └── telegram_tools.py      # send_message, send_with_buttons
├── db/
│   ├── __init__.py
│   ├── supabase_client.py     # Cliente Supabase singleton
│   ├── models.py              # Modelos Pydantic (Tarea, Hallazgo, Sesion)
│   └── queries.py             # Queries SQL reutilizables
├── memory/
│   ├── __init__.py
│   └── redis_session.py       # SessionService custom sobre Upstash Redis
├── schedulers/
│   ├── __init__.py
│   ├── morning_checkin.py     # Rutina 7:30 AM
│   ├── confrontacion.py       # Motor WF-09 cada 6h
│   └── weekly_report.py       # Reporte dominical
├── config/
│   ├── settings.py            # Variables de entorno (Pydantic Settings)
│   └── energy_map.yaml        # Mapa energético (editable sin código)
└── tests/
    ├── test_tools.py          # Tests por herramienta
    └── test_agent.py          # Tests de conversación end-to-end
```

4. CÓDIGO CLAVE — PROMPTS EXACTOS PARA ANTIGRAVITY

Cada sección de abajo es un prompt que se le da a Antigravity para generar el módulo correspondiente. El orden importa: generar en secuencia Sprint 1 → 4.

4.1 Sprint 1 — Módulo 1: Definición del Agente ADK

```
agent/neztask_agent.py
# PROMPT PARA ANTIGRAVITY — Módulo: agent/neztask_agent.py

from google.adk.agents import Agent
from google.adk.models.lite_llm import LiteLlm
from tools.task_tools import create_task, get_tasks, update_task
from tools.hallazgo_tools import save_hallazgo
from tools.planning_tools import get_daily_plan
from tools.calendar_tools import sync_calendar
from tools.audio_tools import transcribe_audio
from tools.telegram_tools import send_telegram
from agent.instructions import SYSTEM_PROMPT

neztask_agent = Agent(
    name='NexTask',
    model='gemini-2.0-flash',
    description='Agente de productividad de Ricardo, Director de Nextstat',
    instruction=SYSTEM_PROMPT,
    tools=[
        create_task,
        get_tasks,
        update_task,
        save_hallazgo,
        get_daily_plan,
        sync_calendar,
        transcribe_audio,
        send_telegram,
    ],
)

# ADK auto-genera el JSON schema de cada Tool desde los type hints
# y el docstring de la función — sin configuración manual adicional
```

4.2 Sprint 1 — Módulo 2: Tool de Captura de Tareas

```
tools/task_tools.py
# PROMPT PARA ANTIGRAVITY — Módulo: tools/task_tools.py

from google.adk.tools import tool
from db.supabase_client import get_client
from db.models import Tarea, CategoriaMacro, CategoriaValor
```

```
from datetime import datetime
from typing import Optional

@tool
def create_task(
    titulo: str,
    categoria_macro: CategoriaMacro,
    categoria_valor: CategoriaValor,
    deadline: Optional[str] = None,
    hora_fija: bool = False,
    personas: list[str] = [],
) -> dict:
    '''Crea una nueva tarea en Supabase y la devuelve con ID asignado.
    Usa esta herramienta cuando el usuario mencione algo que debe hacer,
    un compromiso, una reunión o cualquier acción pendiente.
    Calcula automáticamente el puntaje Pareto antes de guardar.
    '''
    prioridad = calcular_prioridad_pareto(categoria_macro, deadline)
    tarea = Tarea(
        titulo=titulo,
        categoria_macro=categoria_macro,
        categoria_valor=categoria_valor,
        deadline=deadline,
        hora_fija=hora_fija,
        personas=personas,
        prioridad_score=prioridad,
        estado='pendiente',
        creado_en=datetime.utcnow().isoformat(),
    )
    result = get_client().table('tareas').insert(tarea.model_dump()).execute()
    return {'id': result.data[0]['id'], 'titulo': titulo, 'prioridad':
prioridad}

def calcular_prioridad_pareto(
    categoria: CategoriaMacro, deadline: Optional[str]
) -> float:
    impacto  = 5 if categoria == 'operativa' else 2
    urgencia = 5 if deadline and dias_hasta(deadline) <= 1 else 3
    cat_score = 5 if categoria == 'operativa' else 1
    return round((impacto*0.4) + (urgencia*0.3) + (cat_score*0.2), 2)
```

4.3 Sprint 1 — Módulo 3: Memoria Redis (SessionService)

```
memory/redis_session.py
# PROMPT PARA ANTIGRAVITY — Módulo: memory/redis_session.py
# Resuelve el gap crítico de estado conversacional del PRD original

from google.adk.sessions import BaseSessionService, Session
from upstash_redis import Redis
import json, os
from config.settings import settings
```

```
redis = Redis(url=settings.UPSTASH_REDIS_URL,
token=settings.UPSTASH_REDIS_TOKEN)

class RedisSessionService(BaseSessionService):
    '''Almacena el contexto de conversación en Upstash Redis.
    TTL de 24h para evitar contexto obsoleto.
    Clave: session:{app}:{chat_id}
    '''
    TTL_SECONDS = 86400  # 24 horas

    def get_session(self, app_name, user_id, session_id) -> Session:
        key = f'session:{app_name}:{user_id}'
        raw = redis.get(key)
        if raw:
            data = json.loads(raw)
            return Session(**data)
        return Session(app_name=app_name, user_id=user_id,
                       session_id=session_id, state={})

    def save_session(self, session: Session) -> None:
        key = f'session:{session.app_name}:{session.user_id}'
        redis.set(key, session.model_dump_json(), ex=self.TTL_SECONDS)

    def delete_session(self, app_name, user_id, session_id) -> None:
        redis.delete(f'session:{app_name}:{user_id}')

# En runner.py, usarlo así:
# runner = Runner(agent=neztask_agent,
#                 session_service=RedisSessionService(),
#                 app_name='neztask')
```

4.4 Sprint 1 — Módulo 4: Webhook Principal (FastAPI)

main.py — Punto de entrada único

```
# PROMPT PARA ANTIGRAVITY — Módulo: main.py

from fastapi import FastAPI, Request, BackgroundTasks
from agent.runner import procesar_mensaje, procesar_audio
from schedulers.morning_checkin import ejecutar_checkin
from schedulers.confrontacion import ejecutar_confrontacion
from schedulers.weekly_report import ejecutar_reporte_semanal

app = FastAPI(title='NexTask API')

@app.post('/webhook')
async def telegram_webhook(request: Request, bg: BackgroundTasks):
    '''Recibe todos los mensajes de Telegram.',
    data = await request.json()
    message = data.get('message', {})
    chat_id = str(message['chat']['id'])
```

```
    # Detectar tipo de mensaje
    if 'voice' in message or 'audio' in message:
        file_id = message.get('voice', message.get('audio'))['file_id']
        bg.add_task(procesar_audio, chat_id, file_id)
    elif 'text' in message:
        bg.add_task(procesar_mensaje, chat_id, message['text'])

    return {'ok': True}  # Responder a Telegram inmediatamente

@app.post('/scheduler')
async def scheduler_trigger(event: str, bg: BackgroundTasks):
    '''Recibe triggers de Cloud Scheduler para rutinas automáticas.
    eventos: morning_checkin | confrontacion | weekly_report
    '''
    EVENTOS = {
        'morning_checkin': ejecutar_checkin,
        'confrontacion':    ejecutar_confrontacion,
        'weekly_report':    ejecutar_reporte_semanal,
    }
    if handler := EVENTOS.get(event):
        bg.add_task(handler)
    return {'triggered': event}

@app.get('/healthz')
async def health(): return {'status': 'ok'}
```

4.5 Sprint 2 — Check-in Energético Matutino

schedulers/morning_checkin.py

```
# PROMPT PARA ANTIGRAVITY — Módulo: schedulers/morning_checkin.py

from agent.runner import procesar_mensaje
from db.queries import get_pending_tasks, get_yesterday_summary
from config.settings import settings

CHAT_ID = settings.TELEGRAM_CHAT_ID  # ID de Ricardo

async def ejecutar_checkin():
    '''Genera y envía la rutina matutina de las 7:30 AM.
    Obtiene contexto de Supabase y lo inyecta al agente.
    '''
    ayer = await get_yesterday_summary()
    pendientes = await get_pending_tasks(limit=5)

    # Inyectamos el contexto al agente como mensaje del sistema
    contexto = f'''
RUTINA_MATUTINA:
Completadas ayer: {ayer['completadas']}
Pendientes de ayer: {ayer['pendientes']}
Hallazgos de ayer: {ayer['hallazgos']}
```

```
Tareas prioritarias hoy: {[t['titulo'] for t in pendientes]}

Ejecuta la rutina matutina completa según tus instrucciones.
Pide el nivel de energía 1-5 al usuario.
'''
    await procesar_mensaje(CHAT_ID, contexto, es_sistema=True)

# Patrón: el agente recibe el contexto como texto plano.
# NO necesitamos un workflow separado — es solo otra llamada al agente.
```

4.6 Sprint 3 — Motor de Confrontación (WF-09 equivalente)

```
schedulers/confrontacion.py
# PROMPT PARA ANTIGRAVITY — Módulo: schedulers/confrontacion.py

from db.queries import (
    get_overdue_operative_tasks,
    get_operative_ratio_48h,
    get_repeatedly_postponed_tasks,
    get_inactive_value_chains,
)
from agent.runner import procesar_mensaje
from config.settings import settings
from memory.redis_session import RedisSessionService

redis_svc = RedisSessionService()
CHAT_ID  = settings.TELEGRAM_CHAT_ID

async def ejecutar_confrontacion():
    '''Motor WF-09: evalúa patrones y confronta solo si hay evidencia real.
    Respeta modo pausa (/pausa) almacenado en Redis.
    '''
    # Verificar si el usuario activó /pausa
    session = redis_svc.get_session('neztask', CHAT_ID, 'main')
    if session.state.get('pausado_hasta'):
        return  # Silencio activo — no confrontar

    # Evaluar condiciones
    vencidas     = await get_overdue_operative_tasks(horas=24)
    ratio        = await get_operative_ratio_48h()
    postergadas  = await get_repeatedly_postponed_tasks(min_veces=3)
    areas_inact  = await get_inactive_value_chains(dias=3)

    if not any([vencidas, ratio < 0.6, postergadas, areas_inact]):
        return  # Todo bien — no molestar

    # Construir contexto y dejar que el LLM genere el mensaje
    contexto = f'''
EVALUACION_CONFRONTACION:
Tareas operativas vencidas (+24h): {[t['titulo'] for t in vencidas]}
Ratio operativo/experimentación (48h): {ratio:.0%} (objetivo: 70%)
```

```
Tareas postergadas 3+ veces: {[t['titulo'] for t in postergadas]}
Áreas sin actividad 3+ días: {areas_inact}

Genera un mensaje de confrontación directo pero no agresivo.
Máximo 3 puntos. Propón una acción concreta.
'''
    await procesar_mensaje(CHAT_ID, contexto, es_sistema=True)
```

5. BASE DE DATOS — SUPABASE vs NOTION

5.1 Comparativa de Performance

Métrica	Notion API (PRD original)	Supabase PostgreSQL (propuesto)
Latencia promedio	400 – 800 ms por operación	20 – 50 ms por operación
Rate limit	3 req/s (bloqueante en uso intenso)	Sin límite en tier gratuito
Queries complejas	No soportadas (solo filtros simples)	SQL completo, JOINs, agregaciones
Transacciones ACID	No existe	Nativo PostgreSQL
Tier gratuito	Ilimitado (pero lento)	500 MB, ilimitado en filas
Interfaz visual	Excelente (es su fortaleza)	Studio básico (usar Notion para esto)
Rol en la nueva arquitectura	Solo dashboard visual de consulta	BD operacional principal

5.2 Schema SQL para Antigravity

Este schema es el prompt directo para que Antigravity genere el script de migración de Supabase:

db/schema.sql — Schema completo

```
-- PROMPT PARA ANTIGRAVITY — db/schema.sql
-- Ejecutar en Supabase SQL Editor

CREATE TABLE tareas (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  titulo          TEXT NOT NULL,
  categoria_macro TEXT CHECK (categoria_macro IN
('operativa','experimentacion')),
  categoria_valor TEXT,  -- comercial, entrega, finanzas, admin...
  estado          TEXT DEFAULT 'pendiente',
  prioridad_score FLOAT,
  deadline        TIMESTAMPTZ,
  hora_fija       BOOLEAN DEFAULT false,
  bloque_energia  TEXT,  -- alta, media, baja
  personas        TEXT[],
  veces_pospuesta INT DEFAULT 0,
  gcal_event_id   TEXT,
  creado_en       TIMESTAMPTZ DEFAULT NOW(),
  completado_en   TIMESTAMPTZ
);

CREATE TABLE hallazgos (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
  texto       TEXT NOT NULL,
  contexto    TEXT,
  notion_id   TEXT,  -- sincronizado con Notion
  creado_en   TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE config_usuario (
  clave  TEXT PRIMARY KEY,
  valor  TEXT,
  tipo   TEXT  -- string, int, json
);

-- Índices para queries frecuentes del agente
CREATE INDEX idx_tareas_estado    ON tareas(estado);
CREATE INDEX idx_tareas_deadline  ON tareas(deadline);
CREATE INDEX idx_tareas_categoria ON tareas(categoria_macro);

-- Vista para el ratio operativo/experimentacion (WF-09)
CREATE VIEW ratio_48h AS
  SELECT
    COUNT(*) FILTER (WHERE categoria_macro='operativa')     AS operativas,
    COUNT(*) FILTER (WHERE categoria_macro='experimentacion') AS
experimentacion,
    ROUND(COUNT(*) FILTER (WHERE categoria_macro='operativa')::NUMERIC /
          NULLIF(COUNT(*),0) * 100, 1)                      AS pct_operativa
  FROM tareas
  WHERE creado_en > NOW() - INTERVAL '48 hours';
```

6. GUÍA DE IMPLEMENTACIÓN POR SPRINTS PARA ANTIGRAVITY

Esta sección define exactamente qué pedirle a Antigravity en cada sprint, en qué orden, y qué archivos esperar como output. El orden es crítico — cada módulo depende del anterior.

Sprint 1 — MVP Funcional (Días 1-3)

#	Módulo	Prompt para Antigravity	Output esperado
1	config/ settings.py	'Genera un módulo Pydantic Settings que lea: TELEGRAM_BOT_TOKEN, TELEGRAM_CHAT_ID, GEMINI_API_KEY, SUPABASE_URL, SUPABASE_KEY, UPSTASH_REDIS_URL, UPSTASH_REDIS_TOKEN desde variables de entorno.'	settings.py con validación automática de env vars
2	db/ schema.sql	'Genera el schema SQL del punto 5.2 de este documento para Supabase.'	Schema ejecutable en Supabase SQL Editor
3	db/ supabase_client.py	'Genera un cliente Supabase singleton con supabase-py. Incluye retry de 3 intentos con backoff exponencial.'	Cliente reutilizable en todos los tools
4	memory/ redis_session.py	'Genera el RedisSessionService del punto 4.3 de este documento.'	SessionService listo para ADK
5	tools/ task_tools.py	'Genera los tools del punto 4.2: create_task, get_tasks, update_task, complete_task. Usar el schema SQL del paso 2.'	4 tools con type hints y docstrings en español
6	agent/ neztask_agent.py	'Genera el agente ADK del punto 4.1 usando los tools del paso 5.'	Agente funcional con Gemini 2.0 Flash
7	main.py	'Genera el FastAPI del punto 4.4. Recibe webhook de Telegram, detecta texto/audio, delega al agente.'	Webhook funcional. Test: enviar texto a Telegram y ver tarea en Supabase

Sprint 2 — Inteligencia (Días 4-6)

#	Módulo	Prompt para Antigravity	Output esperado
8	tools/ audio_tools.py	'Genera transcribe_audio: descarga el archivo de voz de Telegram via file_id, lo envía a Gemini como audio (multimodal) y retorna el texto transcrito.'	Transcripción nativa en Gemini. Elimina WF-12.

9	tools/planning_tools.py	'Genera get_daily_plan: consulta tareas pendientes de Supabase, las ordena por prioridad_score, las asigna a bloques energéticos según energy_map.yaml y el nivel de energía recibido (1-5).'	Plan del día personalizado por nivel de energía
10	schedulers/morning_checkin.py	'Genera el checkin matutino del punto 4.5: obtiene resumen de ayer desde Supabase e inyecta al agente como contexto.'	Rutina 7:30 AM funcional. Test: invocar manualmente.
11	agent/instructions.py	'Genera el system prompt en español para NexTaskAgent. Incluir: personalidad directa, reglas Pareto 70/30, cómo confrontar, cómo manejar energía 1-5, formato de respuesta en Telegram (sin Markdown).'	System prompt versionable separado del código

Sprint 3 — Proactividad (Días 7-9)

#	Módulo	Prompt para Antigravity	Output esperado
12	schedulers/confrontacion.py	'Genera el motor de confrontación del punto 4.6. Importante: verificar estado /pausa en Redis antes de enviar. Usar las queries de ratio_48h de Supabase.'	WF-09 completo. Respeta pausa. Test: insertar tareas vencidas en Supabase.
13	tools/telegram_tools.py	'Genera send_telegram con soporte de botones inline. Incluir comando /pausa que guarda estado en Redis con TTL configurable (default 2h).'	Botones en mensajes + /pausa funcional
14	schedulers/weekly_report.py	'Genera el reporte semanal dominical. Consultar Supabase: % completadas, ratio real operativo/exp, tareas postergadas 3+ veces, resumen de hallazgos.'	Reporte semanal automático con métricas reales

Sprint 4 — Optimización (Días 10-12)

#	Módulo	Prompt para Antigravity	Output esperado
15	tools/calendar_tools.py	'Genera sync_calendar bidireccional: Supabase → Google Calendar (crear/actualizar evento) y webhook GCal → Supabase. Usar colores por categoría.'	Sincronización bidireccional funcional
16	tools/hallazgo_to	'Genera save_hallazgo: guarda en Supabase Y sincroniza con Notion API	Hallazgos en Supabase + visibles en Notion

	ols.py	como fuente de visualización de hallazgos.'	
17	tests/	'Genera tests pytest para: create_task, get_daily_plan, RedisSessionService, confrontacion con estado pausado, y un test e2e de conversación completa.'	Suite de tests. CI/CD listo para GitHub Actions.
18	Dockerfile + deploy	'Genera Dockerfile multi-stage para Cloud Run. Incluir: uvicorn, variables de entorno desde Secret Manager, health check en /healthz.'	Deploy 1-click a Cloud Run

7. RESUMEN COMPARATIVO: n8n vs STACK PROPUESTO

Criterio	n8n (PRD original)	Google ADK + Supabase + Redis
Trabajo manual de setup	❌ Alto — 14 workflows a mano nodo por nodo	✅ Bajo — Antigravity genera 18 módulos Python
Memoria conversacional	❌ Sin solución — requiere workaround Notion	✅ Nativa en SessionService + Redis TTL 24h
Estado entre mensajes	❌ Perdido entre ejecuciones (stateless)	✅ Persistido automáticamente en Redis
Latencia total por mensaje	⚠️ 2-4s (Notion 400ms + n8n + Gemini)	✅ 800ms-1.5s (Supabase 30ms + ADK + Gemini)
Facilidad para Antigravity	❌ JSON workflow propietario, difícil de generar	✅ Python modular — Antigravity genera módulo por módulo
Costo infra mensual	✅ $0 self-hosted	✅ $0 (todos free tier)
Debugging	⚠️ Visual nodo por nodo, sin stack trace	✅ Stack traces Python, logs estructurados, pytest
Escalabilidad multi-usuario	❌ Arquitectura single-tenant, difícil extender	✅ Supabase RLS + Redis keys por usuario — multi-tenant desde Sprint 1
Transcripción de audio	❌ WF-12 separado, servicio externo no definido	✅ Gemini multimodal nativo — 0 costo adicional
Tiempo estimado MVP	⚠️ 5-7 días (configuración manual)	✅ 2-3 días (Antigravity genera, humano revisa)
Potencial expansión a clientes	❌ Muy limitado — arquitectura no diseñada para ello	✅ Agregar chat_id de nuevo cliente = nuevo tenant en 10 min

8. CONCLUSIÓN Y PRÓXIMOS PASOS

✅ **Stack recomendado (definitivo)**

Google ADK + Gemini 2.0 Flash + Supabase PostgreSQL + Upstash Redis + FastAPI en Cloud Run. Costo: $0/mes. Tiempo de MVP: 2-3 días con Antigravity. Elimina los 14 workflows manuales de n8n. Resuelve los 4 gaps críticos del PRD original.

Próximos Pasos Inmediatos

Paso	Acción	Detalle	Tiempo
1	Crear proyecto Supabase	supabase.com → New Project → ejecutar schema.sql del punto 5.2	**15 min**
2	Crear cuenta Upstash	upstash.com → New Redis Database → copiar URL y TOKEN	**5 min**
3	Abrir Antigravity	Dar como contexto: este documento + el PRD original. Pedir Sprint 1 módulo por módulo en el orden de la sección 6.	**1-2 horas**
4	Deploy en Cloud Run	gcloud run deploy neztask --source . → copiar URL al webhook de Telegram Bot API	**20 min**
5	Test end-to-end Sprint 1	Enviar texto a Telegram → verificar tarea en Supabase → verificar evento en Google Calendar	**30 min**
6	Configurar Cloud Scheduler	3 jobs: morning_checkin (7:30), confrontacion (c/6h), weekly_report (dom 18:00)	**10 min**

Nextstat — Datos claros. Decisiones seguras. — Santa Cruz de la Sierra, Bolivia — Marzo 2026